

Projekt do predmetu ISA - Filtrujúci DNS Resolver

Filip Botlo - xbotlo01

29. september 2025

Obsah

1	Úvod do problematiky	2
2	Návrh aplikácie	2
2.1	Architektúra systému	2
3	Implementácia	2
3.1	Prehľad tried a ich funkcionality	2
3.1.1	Trieda CLIParser	2
3.1.2	Trieda DNSResolver	3
3.1.3	Trieda DNSProtocol	3
3.2	DNS protokol implementácia	3
3.3	Filtrovacia logika	3
3.4	Rozšírenia aplikácie	4
3.4.1	Verbose mód	4
3.4.2	Štatistiky	4
4	Testovanie	4
4.0.1	Základné testy s verbose módom	4
4.1	Testovací Python skript	6
4.1.1	Základné testy	6
4.1.2	Testy argumentov	6
4.1.3	Testy protokolov	6
4.1.4	Testy súborov	6
4.1.5	DNS dotazy	6
4.1.6	Chybové scenáre	7
4.1.7	Výsledky	7
4.2	Výsledky testov	7
5	Záver	7
6	Literatúra a zdroje	7

1 Úvod do problematiky

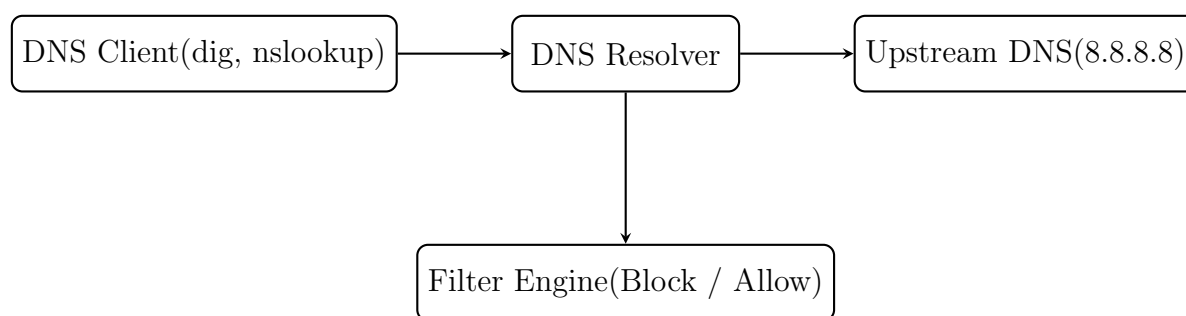
DNS (Domain Name System) je hierarchický distribuovaný systém pre preklad doménových mien na IP adresy a naopak. Je základným stavebným kameňom internetu, ktorý umožňuje používateľom pristupovať k webovým stránkam pomocou ľahko zapamätateľných názvov namiesto číselných IP adries.

Cieľom projektu bolo implementovať vlastný DNS resolver, ktorý spracúva DNS dotazy a aplikuje na ne filtrovaciu logiku.

2 Návrh aplikácie

2.1 Architektúra systému

Aplikácia je navrhnutá ako modulárny systém s jasne oddelenými zodpovednosťami. Používateľ (klient) posieľa DNS dotaz napríklad cez nástroje **dig** alebo **nslookup**. Dotaz prichádza do DNS resolvera, ktorý ho analyzuje a rozhodne, či je možné ho odoslať na DNS server (napr. Google DNS 8.8.8.8), alebo ho treba blokovať podľa pravidiel filtra.



Obr. 1: Architektúra systému

3 Implementácia

3.1 Prehľad tried a ich funkcionality

Aplikácia je implementovaná v jazyku C++ s využitím objektovo orientovaného programovania. Hlavné triedy sú navrhnuté tak, aby každá mala jasne definovanú zodpovednosť a minimalizovala závislosti medzi modulmi.

3.1.1 Trieda CLIParser

Trieda je zodpovedná za spracovanie argumentov príkazového riadka. Obsahuje metódy pre parsovanie jednotlivých parametrov, validáciu vstupných hodnôt a generovanie nápovedy. Hlavné funkcie tejto triedy zahŕňajú:

- `parse()` - hlavná metóda pre spracovanie argumentov
- `validateArguments()` - kontrola platnosti zadaných parametrov
- `printHelp()` - zobrazenie nápovedy pre používateľa

Trieda podporuje všetky potrebné parametre ako port, upstream DNS server, cestu k filtrovaciemu súboru a verbose mód. Implementuje error handling pre neplatné vstupy.

3.1.2 Trieda DNSResolver

Trieda predstavuje jadro aplikácie a koordinuje všetky operácie. Je zodpovedná za vytvorenie sieťových socketov, implementáciu serverového cyklu a správu komunikácie medzi klientmi a upstream DNS serverom. Kľúčové metódy tejto triedy:

- `createClientSocket()` - vytvorenie socketu pre prijímanie dotazov od klientov
- `createResolverSocket()` - vytvorenie socketu pre komunikáciu s upstream DNS
- `loadFilterFile()` - načítanie zoznamu blokováných domén zo súboru
- `handleQuery()` - spracovanie jednotlivého DNS dotazu
- `run()` - hlavný serverový cyklus

Serverový cyklus používa `select()` s `recvfrom()` pre prijímanie dotazov a `sendto()` pre odosielanie odpovedí.

3.1.3 Trieda DNSProtocol

Trieda `DNSProtocol` obsahuje všetku logiku súvisiacu s DNS protokolom podľa RFC 1035. Je implementovaná ako statická trieda s utility metódami pre prácu s DNS správami. Hlavné funkcie:

- `parseDomain()` - parsovanie doménových mien z DNS správ
- `isQuery()` - kontrola, či je správa DNS dotazom
- `processQuery()` - hlavná logika filtrovania a spracovania dotazov
- `sendErrorResponse()` - generovanie chybových DNS odpovedí
- `getClientAddressString()` - konverzia adresy na čitateľný formát

3.2 DNS protokol implementácia

Parsovanie doménových mien je jednou z najzložitejších častí implementácie. Domény sú uložené ako sekvencia labelov, kde každý label má svoju dĺžku nasledovanú obsahom. Resolver dokáže správne dekodovať tieto štruktúry a rekonštruovať plné doménové meno.

3.3 Filtrovacia logika

Najskôr sa doména normalizuje - prevedie sa na malé písmená a odstránia sa nadbytočné znaky ako koncové bodky. Následne sa kontroluje presná zhoda s blokovými doménami.

Pre podporu subdomén sa implementuje algoritmus, ktorý kontroluje, či normalizovaná doména končí bodkou nasledovanou niektorou z blokováných domén. Toto umožňuje blokovat' všetky poddomény bez potreby explicitne uvádzať každú z nich.

Filtrovacia logika podporuje rôzne typy DNS záznamov, pričom A záznamy sú plne podporované. Ostatné typy záznamov sú odmietnuté s chybovým kódom NOTIMP.

3.4 Rozšírenia aplikácie

3.4.1 Verbose mód

Verbose mód je implementovaný ako voliteľný parameter `-v` alebo `--verbose`. Pri aktivácii tohto módu resolver vypisuje podrobné informácie o každom spracovanom DNS dotaze, čo značne uľahčuje ladenie a monitorovanie činnosti aplikácie.

Implementácia verbose módu je integrovaná do triedy `DNSResolver` a `DNSProtocol`, kde sa kontroluje flag `verbose_` pred každým výpisom diagnostických informácií.

3.4.2 Štatistiky

Rozšírenie štatistík je implementované ako voliteľný parameter `--stats`. Toto rozšírenie umožňuje zbieranie a zobrazenie štatistík o činnosti DNS resolvera na konci jeho behu.

Štatistiky zahŕňajú nasledujúce metriky:

- Celkový počet dotazov
- Blokované dotazy
- Preposlané dotazy
- Ostatné typy dotazov (NOTIMP)
- Presné zhody
- Zhody s poddomenami

Implementácia štatistík využíva štruktúru `Statistics` definovanú v `DNSProtocol.hpp`.

4 Testovanie

4.0.1 Základné testy s verbose módom

```
# Test povolenej domény
dig @127.0.0.1 -p 5353 google.com
```

```
Výsledok
=== NOVY DNS DOTAZ ===
ID: 31883
QR: 0 (0=dotaz, 1=odpoved)
QDCOUNT: 1
Dotaz #1: google.com (typ: 1)
POVOLENE: google.com
Preposielam dotaz na resolver 8.8.8.8
Prijata odpoved od resolvera (55 bytov)
Odpoved odoslana klientovi
=== KONIEC DOTAZU ===
```

```
# Test blokovanej domény
dig @127.0.0.1 -p 5353 facebook.com
```

```
#Výsledok
=== NOVY DNS DOTAZ ===
ID: 15954
Dotaz: facebook.com (typ: 1)
PRESNA ZHODA: facebook.com == facebook.com
BLOKOVANE: facebook.com
Odosielam REFUSED odpoved
Error response sent (code: 5)
```

```
# Test IPv6
dig @::1 -p 5353 google.com A
```

```
#Výsledok
=== NOVY DNS DOTAZ ===
ID: 2886
QR: 0 (0=dotaz, 1=odpoved)
QDCOUNT: 1
Dotaz #1: google.com (typ: 1)
POVOLENE: google.com
Preposielam dotaz na resolver 8.8.8.8
Prijata odpoved od resolvera (55 bytov)
Odpoved odoslana klientovi
=== KONIEC DOTAZU ===
```

```
# Test rôznych typov záznamov
dig @127.0.0.1 -p 5353 google.com MX
dig @127.0.0.1 -p 5353 google.com AAAA
```

```
#Výsledok
=== NOVY DNS DOTAZ ===
ID: 65002
QR: 0 (0=dotaz, 1=odpoved)
QDCOUNT: 1
Dotaz #1: google.com (typ: 15)
NESPRAVNY TYP: google.com (typ 15, povoleny len A=1)
Odosielam NOTIMP odpoved
Error response sent (code: 4)
Received query from 127.0.0.1
```

```
=== NOVY DNS DOTAZ ===
ID: 49103
QR: 0 (0=dotaz, 1=odpoved)
QDCOUNT: 1
Dotaz #1: google.com (typ: 28)
NESPRAVNY TYP: google.com (typ 28, povoleny len A=1)
Odosielam NOTIMP odpoved
Error response sent (code: 4)
```

Received query from 192.168.0.58

4.1 Testovací Python skript

Testovací skript `simple_test.py` obsahuje nasledujúce kategórie testov:

4.1.1 Základné testy

- Kompilácia projektu (`make clean, make`)
- Zobrazenie nápovedy (`--help`)
- Validácia argumentov (chýbajúce, duplicitné, neplatné)

4.1.2 Testy argumentov

- Rôzne poradia argumentov (`-s -f -p, -f -s -p`, atď.)
- Validácia portov (0-65535, neplatné formáty)
- Edge cases (prázdne reťazce, medzery, neplatné IP)

4.1.3 Testy protokolov

- IPv4 a IPv6 DNS servery
- Doménové mená DNS serverov (`dns.google, 1.1.1.1`)
- Localhost testy (`127.0.0.1, ::1`)

4.1.4 Testy súborov

- Rôzne formáty koncov riadkov (Linux, Windows, Mac)
- Súbory s komentármi a prázdnyimi riadkami
- Case sensitivity a medzery v doménach

4.1.5 DNS dotazy

- Blokované domény → `REFUSED`
- Povolené domény → `NOERROR`
- Typy dotazov (len `A` povolené, ostatné → `NOTIMP`)
- Subdomény a case sensitivity

4.1.6 Chybové scenáre

- Neexistujúce súbory a DNS servery
- Prázdne súbory a práva prístupu
- Timeout a exception handling

4.1.7 Výsledky

- Automatické spúšťanie DNS filtra v pozadí
- Monitoring portov a procesov
- Súhrnné štatistiky (počet testov, úspešnosť, chyby)
- Automatické čistenie testovacích súborov

4.2 Výsledky testov

Výsledky ukázali, že resolver úspešne zvládol všetkých 92 testov.

5 Záver

Vďaka projektu som sa naučil pracovať s DNS protokolom na nízkej úrovni a lepšie pochopil jeho štruktúru podľa RFC 1035. Získal som praktické skúsenosti so sieťovým programovaním v C++ a s implementáciou vlastného resolvera.

6 Literatúra a zdroje

1. <https://tools.ietf.org/html/rfc1035>
2. <https://man7.org/linux/man-pages/man2/socket.2.html>
3. <https://man7.org/linux/man-pages/man2/sendto.2.html>
4. <https://man7.org/linux/man-pages/man7/ipv6.7.html>